TRACK A DEMO EVIDENCE

KijaniKiosk release path - screenshot sequence

Dalton Wambui | 2026-11-14 | repository: feature/capstone-track-a

What this is: a sequence of live terminal and Jenkins screenshots captured from the lab environment. It is not a Loom recording. Each page names the claim, the command, and the evidence file.

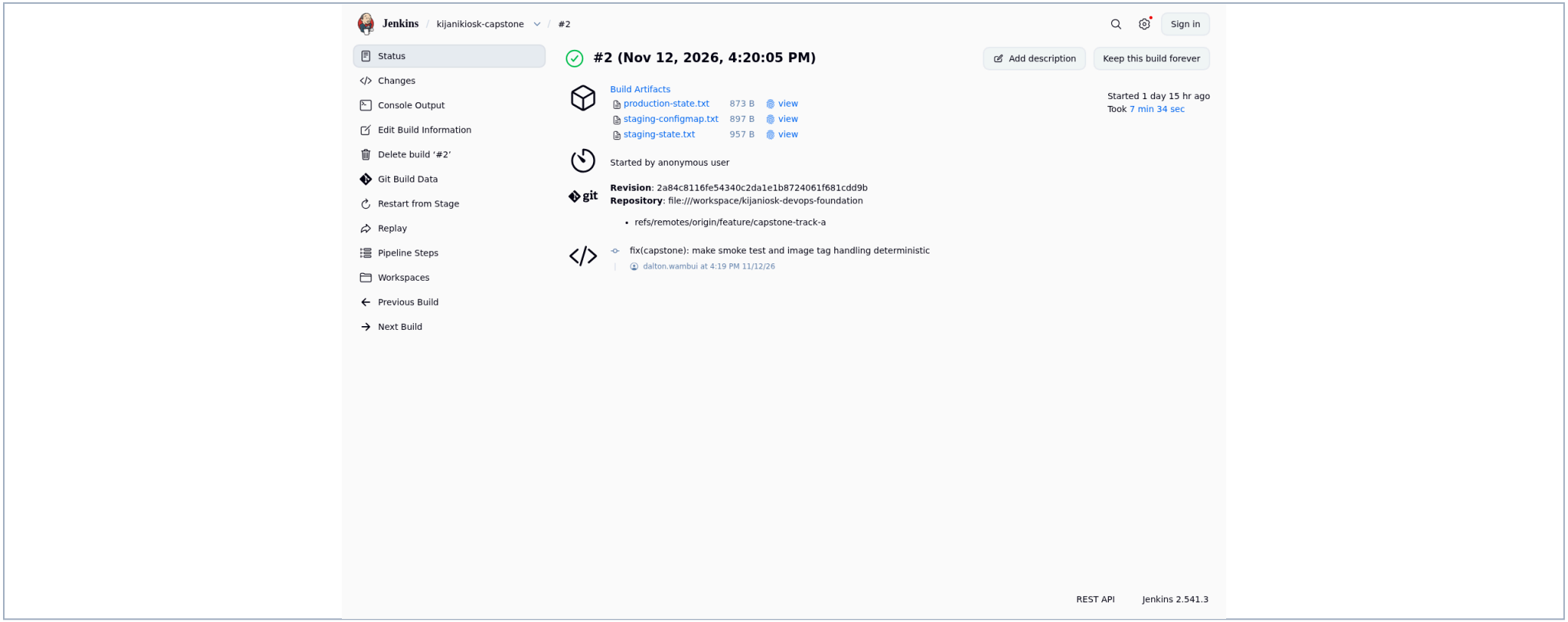
Claims covered: environment separation, version-aware smoke testing, a negative-check failure, pod self-healing, production health, and the Jenkins approval path.

Gaps still shown honestly: Week 10 serverless receipt chain is not implemented; alerts are not evaluated; production is a namespace in the same minikube cluster; peer review is a self-review fallback.

Full logs: `capstone/evidence/demo/` and `capstone/evidence/jenkins-build-2/`.

1. JENKINS PIPELINE

Build #2 completed the staging-to-production path



The build page shows the stages that matter: Verify, Build image, Push image, Deploy staging, Smoke test staging, Approve production, Deploy production, and Smoke test production.

```
Smoke test staging: PASS health version 2.4.0-2a84c81
Approve production: Deploy 2.4.0-2a84c81 to production?
Smoke test production: PASS health version 2.4.0-2a84c81
Finished: SUCCESS
```

Evidence: `capstone/evidence/jenkins-build-2/console.txt`; screenshot: `capstone/evidence/screenshots/jenkins-build-2.png`.

2. APPROVAL GATE AND PRODUCTION PROOF

The gate paused, then production served the same tag

```
=== Jenkins build #2 approval and production proof (excerpt) ===
[Pipeline] { (Smoke test staging)
PASS health: {"service": "kk-payments", "status": "ok", "version": "2.4.0-2a84c81"}
PASS total: {"total": 42}
PASS smoke test: kijani-staging/kk-payments-service
[Pipeline] { (Approve production)
Deploy 2.4.0-2a84c81 to production?
[Pipeline] { (Smoke test production)
PASS health: {"service": "kk-payments", "status": "ok", "version": "2.4.0-2a84c81"}
PASS total: {"total": 42}
PASS smoke test: kijani-production/kk-payments-service
Finished: SUCCESS
```

> Build #2 used the original gate. The current pipeline adds `APPROVER` and `APPROVAL_REASON` parameters and writes `capstone/evidence/approval.txt`. The refreshed build is the source of that file; do not claim it exists until the build has run.

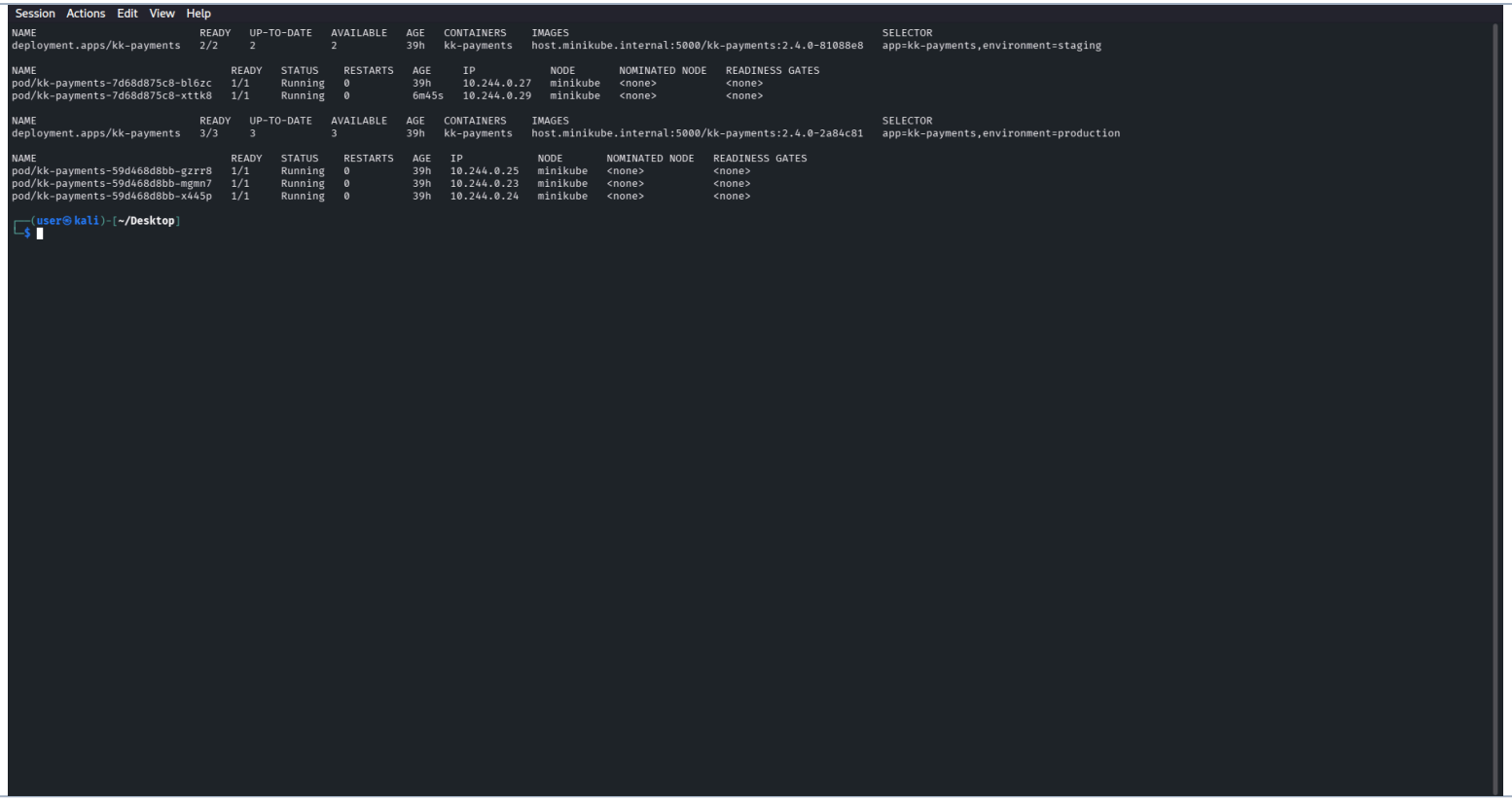
Evidence: `capstone/evidence/demo/approval-build-2.txt`; refreshed approval evidence: `capstone/evidence/approval.txt`.

3. ENVIRONMENT SEPARATION

Same workload, different environment facts



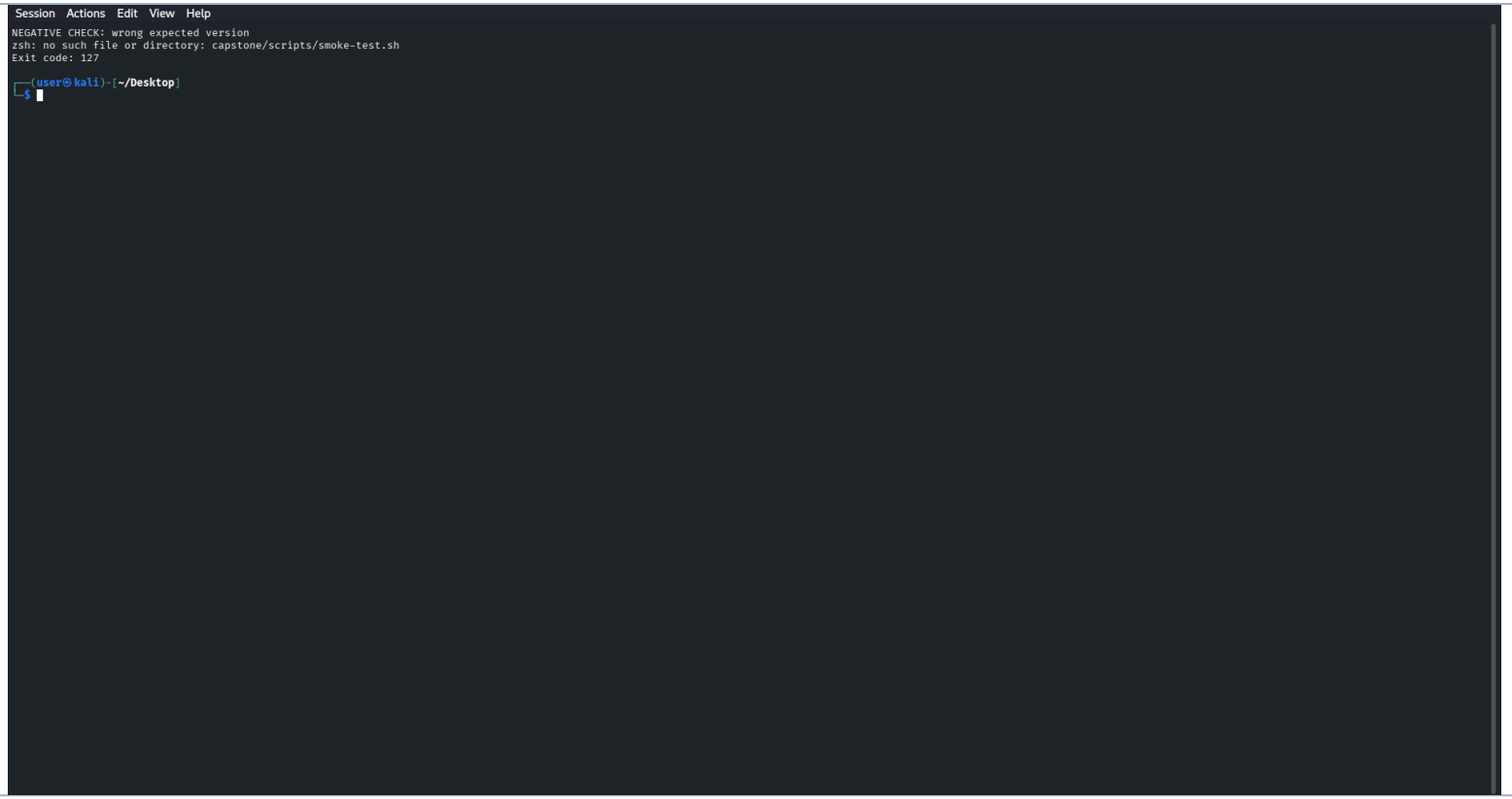
Staging and production render different `DB_HOST`, `NODE_ENV`, `RECEIPTS_BUCKET`, and replica counts. `APP_VERSION` is absent because it belongs to the image, not the ConfigMap.



Evidence: `capstone/evidence/demo/config-render.txt`; screenshot: `capstone/evidence/screenshots/terminal-config.png`, `terminal-cluster-state.png`.

4. NEGATIVE CHECK

A wrong expected version must fail, not pass quietly

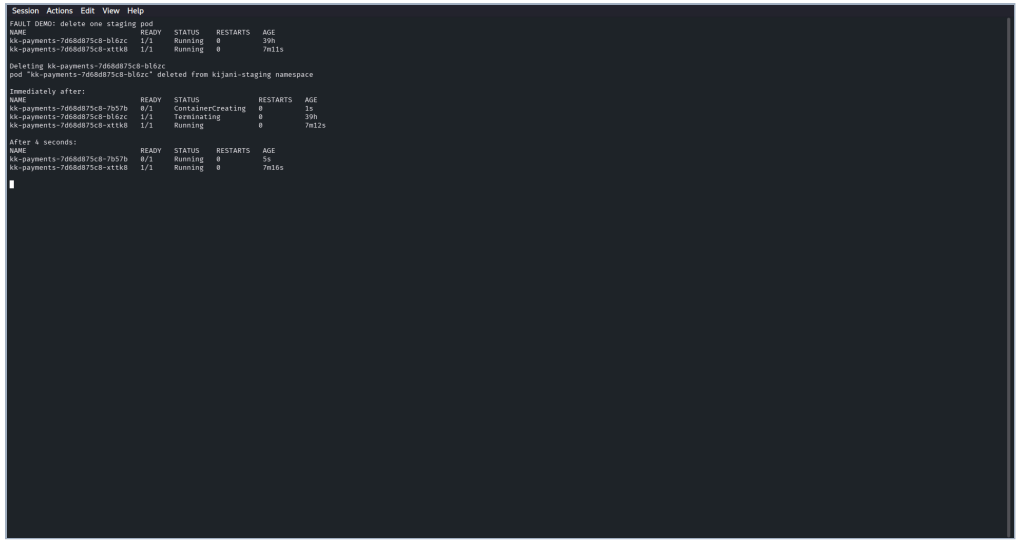


The smoke test exits non-zero and the assertion shows the actual version versus the deliberately wrong expected version. No `PASS` line is printed. This is the guardrail that build #1 exposed as necessary.

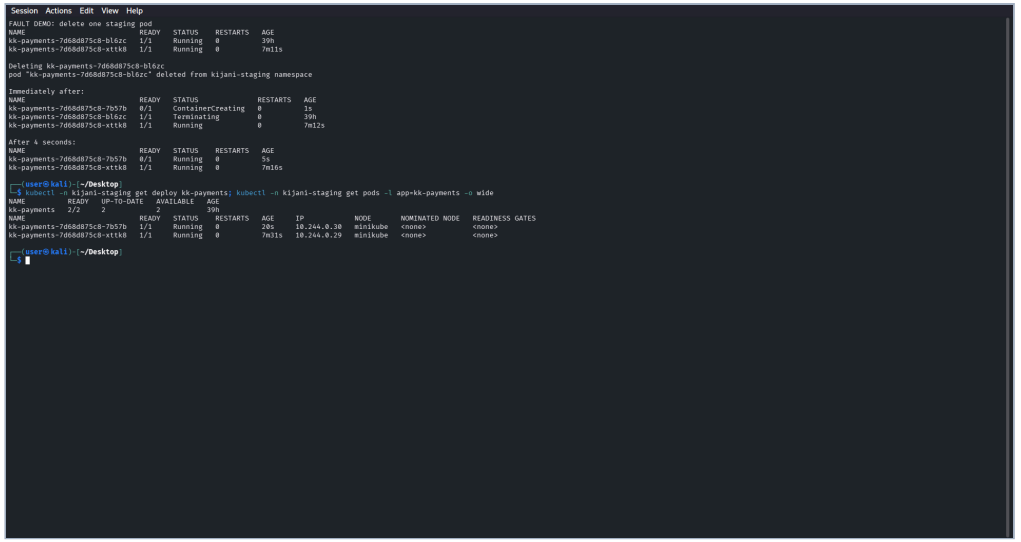
Evidence: `capstone/evidence/demo/negative-smoke.txt`; screenshot: `capstone/evidence/screenshots/terminal-negative-smoke.png`.

5. FAULT HANDLING

Delete a pod; the Deployment replaces it



One pod is Terminating while the replacement is created. The Service still has the other ready pod.

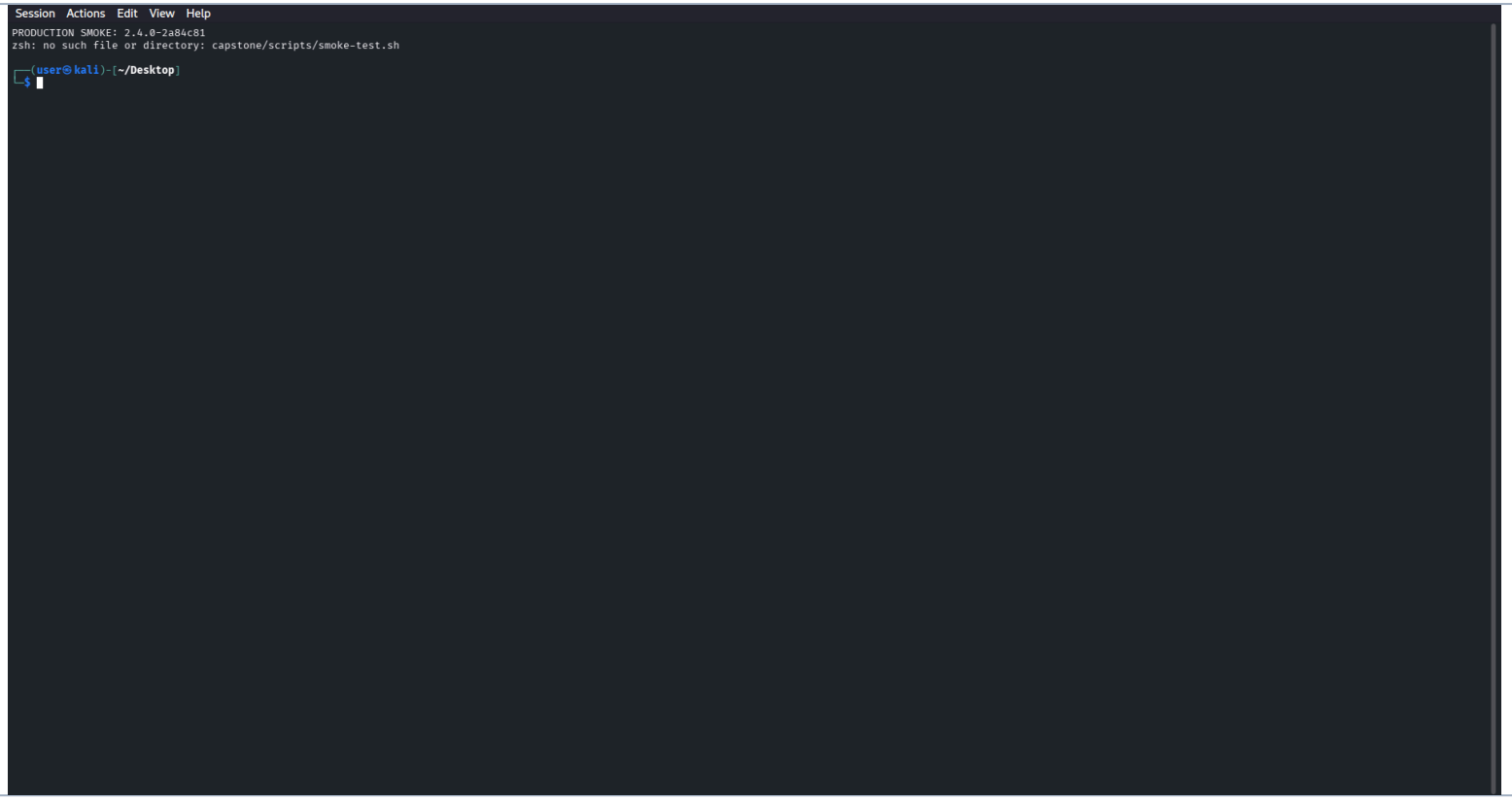


The Deployment returns to 2/2 available without a manual rollout. This is controller self-healing, not a rollback.

Evidence: `capstone/evidence/demo/fault-handling.txt`; screenshots: `terminal-fault-handling.png`, `terminal-fault-handling-after.png`.

6. PRODUCTION PROOF AND HONEST GAPS

Production answers, and the limits are explicit



> **Do not overclaim:** production is a namespace in the same minikube cluster. The PrometheusRule is validated but not evaluated. Week 10 receipt delivery is absent. The approval reason record belongs to the refreshed build, not build #2.

Evidence: `capstone/evidence/demo/` ; claims and commands: `docs/demo-script.md` .